

1. INTRODUCTION TO NLP

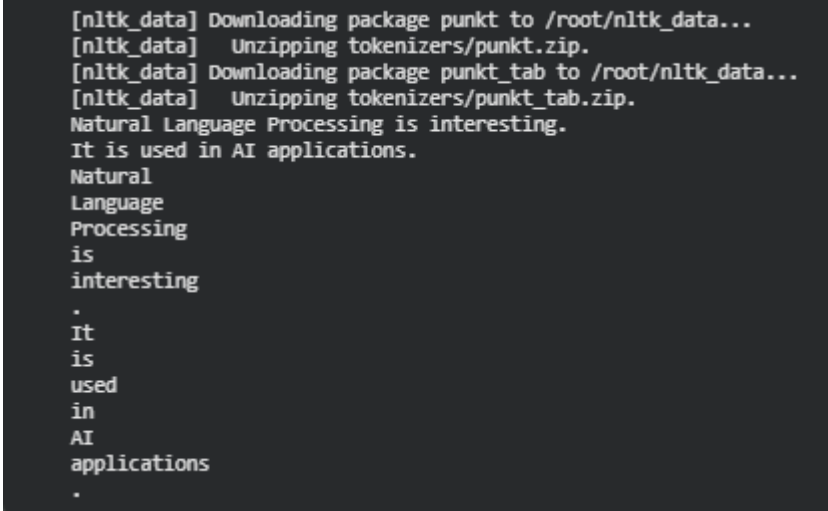
a. Tokenization of Sentences and Words using NLTK and spaCy

(i) Using NLTK:

Input:

```
import nltk
nltk.download('punkt')
from nltk.tokenize import sent_tokenize, word_tokenize
text="Natural Language Processing is interesting. It is used in AI applications."
nltk.download('punkt_tab')
sentences=sent_tokenize(text)
for sentence in sentences:
    print(sentence)
words=word_tokenize(text)
for word in words:
    print(word)
```

Output:

A terminal window with a dark background showing the output of the NLTK code. It displays the download progress for 'punkt' and 'punkt_tab' packages, followed by the tokenized sentences and words.

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
Natural Language Processing is interesting.
It is used in AI applications.
Natural
Language
Processing
is
interesting
.
It
is
used
in
AI
applications
.
```

(ii) Using spaCy:

Input:

```
import spacy #for real applications (in-built features for pos tagging)
nlp=spacy.load('en_core_web_sm')
doc=nlp(text)
for sent in doc.sents:
```

```
print(sent.text)
for token in doc:
    print(token.text)
```

Output:

```
... Natural Language Processing is interesting.
    It is used in AI applications.
    Natural
    Language
    Processing
    is
    interesting
    .
    It
    is
    used
    in
    AI
    applications
    .
```

b. Stemming and Lemmatization on sample text.

(i) Stemming:

Input:

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
nltk.download('punkt') #pretrained model to know prefix and suffix (for pos tagging)
text="The children are running faster than their friends . We are playing badminton ."
nltk.download('punkt_tab')
tokens=word_tokenize(text)
stemmer=PorterStemmer() #function imported by library
stemmed_words = [stemmer.stem(word) for word in tokens]
print(stemmed_words)
```

Output:

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
    [nltk_data]   Unzipping tokenizers/punkt.zip.
    [nltk_data] Downloading package punkt_tab to /root/nltk_data...
    [nltk_data]   Unzipping tokenizers/punkt_tab.zip.
    ['the', 'children', 'are', 'run', 'faster', 'than', 'their', 'friend', '.', 'we', 'are', 'play', 'badminton', '.']
```

(ii) Lemmatization:

- **Using NLTK:**

Input:

```
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
nltk.download('omw-1.4')
lemmatizer= WordNetLemmatizer()
lemmatized_word = [lemmatizer.lemmatize(word) for word in tokens]
print(lemmatized_word)
```

Output:

```
... [nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
['The', 'child', 'are', 'running', 'faster', 'than', 'their', 'friend', '.', 'We', 'are', 'playing', 'badminton', '.']
```

- **Using spaCy:**

Input:

```
import spacy
nlp=spacy.load('en_core_web_sm')
doc=nlp(text)
lemmatized_words = [token.lemma_ for token in doc]
print(lemmatized_words)
```

Output:

```
... ['the', 'child', 'be', 'run', 'fast', 'than', 'their', 'friend', '.', 'we', 'be', 'play', 'badminton', '.']
```

c. Stop-word Removal from a document

Input:

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('punkt')
nltk.download('stopwords')
document="This is a simple document for explaining stop word removal in NLP."
nltk.download('punkt_tab')
tokens=word_tokenize(document)
print(tokens)
stop_words=set(stopwords.words('english'))
filtered_words=[]
```

for word in tokens:

 if word.lower() not in stop_words and word.isalpha():

 filtered_words.append(word)

print(filtered_words)

Output:

```
*** ['This', 'is', 'a', 'simple', 'document', 'for', 'explaining', 'stop', 'word', 'removal', 'in', 'NLP', '.']
['simple', 'document', 'explaining', 'stop', 'word', 'removal', 'NLP']
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

d. Part-of-Speech (POS) Tagging of a given sentence

(i) Using Spacy

Input:

```
import spacy
```

```
# Load the spaCy English model (assuming it's already downloaded)
```

```
try:
```

```
    nlp = spacy.load('en_core_web_sm')
```

```
except OSError:
```

```
    print("Downloading spaCy model 'en_core_web_sm'...")
```

```
    !python -m spacy download en_core_web_sm
```

```
    nlp = spacy.load('en_core_web_sm')
```

```
sentence = "The quick brown fox jumps over the lazy dog."
```

```
# Process the sentence with spaCy
```

```
doc = nlp(sentence)
```

```
print("Original Sentence:", sentence)
```

```
print("\nPOS Tags (spaCy):")
```

```
for token in doc:
```

```
    print(f"{token.text:<10} {token.pos_:<10} {token.tag_:<10}
```

```
    {spacy.explain(token.tag_)})")
```

Output:

```

*** Original Sentence: The quick brown fox jumps over the lazy dog.

POS Tags (spaCy):
The      DET      DT      determiner
quick    ADJ       JJ      adjective (English), other noun-modifier (Chinese)
brown    ADJ       JJ      adjective (English), other noun-modifier (Chinese)
fox       NOUN      NN      noun, singular or mass
jumps    VERB      VBZ     verb, 3rd person singular present
over     ADP        IN      conjunction, subordinating or preposition
the       DET      DT      determiner
lazy     ADJ       JJ      adjective (English), other noun-modifier (Chinese)
dog       NOUN      NN      noun, singular or mass
.         PUNCT      .       punctuation mark, sentence closer

```

(ii) Using NLTK

Input:

```

import nltk
from nltk.tokenize import word_tokenize
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng') # Added to resolve the
specific LookupError
sentence = "The quick brown fox jumps over the lazy dog."
# Tokenize the sentence
words = word_tokenize(sentence)
# Perform POS tagging
pos_tags_nltk = nltk.pos_tag(words)
print("Original Sentence:", sentence)
print("\nPOS Tags (NLTK):")
print(pos_tags_nltk)

```

Output:

```

*** [nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
Original Sentence: The quick brown fox jumps over the lazy dog.

POS Tags (NLTK):
[('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN'), ('.', '.')]

```

e. Parsing and chunking using regex and spaCy

- Parsing and Chunking using NLTK (RegEx)

Input:

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('averaged_perceptron_tagger', quiet=True)
sentence = "The quick brown fox jumps over the lazy dog."
# Tokenize and POS tag the sentence
words = word_tokenize(sentence)
pos_tags = nltk.pos_tag(words)
print("Original Sentence with POS Tags:", pos_tags)
# Define a grammar for noun phrase chunking using regex
grammar = r"""
    NP: {<DT|JJ|NN.*>+}
    """
# Create a RegexpParser with the grammar
cp = nltk.RegexpParser(grammar)
# Apply the parser to the POS-tagged sentence
tree = cp.parse(pos_tags)
print("\nNoun Phrase Chunks (NLTK RegEx):")
for subtree in tree.subtrees():
    if subtree.label() == 'NP':
        print(subtree)
# You can also visualize the tree
# tree.draw() # Uncomment to draw the parse tree (requires matplotlib and
ghostscript)
```

```
... Original Sentence with POS Tags: [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN'), ('.', '.')]

Noun Phrase Chunks (NLTK RegEx):
(NP The/DT quick/JJ brown/NN fox/NN)
(NP the/DT lazy/JJ dog/NN)
```

● Parsing and Chunking using spaCy

Input:

```
import spacy
# Load the spaCy English model
try:
    nlp = spacy.load('en_core_web_sm')
except OSError:
    print("Downloading spaCy model 'en_core_web_sm'...")
    !python -m spacy download en_core_web_sm
    nlp = spacy.load('en_core_web_sm')
sentence = "The quick brown fox jumps over the lazy dog."
```

```
# Process the sentence with spaCy
doc = nlp(sentence)
print("Original Sentence:", sentence)
print("\nNoun Chunks (spaCy):")
for chunk in doc.noun_chunks:
    print(chunk.text, "---", chunk.root.text, "---", chunk.root.dep_, "---",
          chunk.root.head.text)
```

```
... Original Sentence: The quick brown fox jumps over the lazy dog.

Noun Chunks (spaCy):
The quick brown fox --- fox --- nsubj --- jumps
the lazy dog --- dog --- pobj --- over
```

f. Name entity recognition(NER) using spaCy

Input:

```
import spacy
```

```
# Load the spaCy English model (assuming it's already downloaded)
try:
```

```
    nlp = spacy.load('en_core_web_sm')
```

```
except OSError:
```

```
    print("Downloading spaCy model 'en_core_web_sm'...")
```

```
    !python -m spacy download en_core_web_sm
```

```
    nlp = spacy.load('en_core_web_sm')
```

```
sentence = "Apple Inc. was founded by Steve Jobs, Steve Wozniak, and Ronald
Wayne in 1976. Its headquarters are in Cupertino, California."
```

```
# Process the sentence with spaCy
```

```
doc = nlp(sentence)
```

```
print("Original Sentence:", sentence)
```

```
print("\nNamed Entities (spaCy):")
```

```
for ent in doc.ents:
```

```
    print(f'{ent.text:<20} {ent.label_:<15} {spacy.explain(ent.label_)}')
```

```
... Original Sentence: Apple Inc. was founded by Steve Jobs, Steve Wozniak, and Ronald Wayne in 1976. Its headquarters are in Cupertino, California.

Named Entities (spaCy):
Apple Inc.      ORG      Companies, agencies, institutions, etc.
Steve Jobs     PERSON   People, including fictional
Steve Wozniak  PERSON   People, including fictional
Ronald Wayne   PERSON   People, including fictional
1976           DATE     Absolute or relative dates or periods
Cupertino      GPE      Countries, cities, states
California     GPE      Countries, cities, states
```

g. Coreference resolution using neuralcoref(or similar)

Input:

```
import spacy
# import coreferee # Commented out due to Python version incompatibility with
coreferee

# Load the spaCy English model (preferably large for coreference)
try:
    nlp = spacy.load('en_core_web_lg')
except OSError:
    print("Downloading spaCy model 'en_core_web_lg'...")
    !python -m spacy download en_core_web_lg
    nlp = spacy.load('en_core_web_lg')

# The coreferee pipeline component cannot be added because coreferee is not
compatible with Python 3.12.
# nlp.add_pipe('coreferee')

sample_text = "John met Mary. He told her that he was going to the store. She smiled
at him."

# Process the text with spaCy
doc = nlp(sample_text)

print("Original Text:", sample_text)
print("\nCoreference Clusters (coreferee):")
# Coreference resolution using coreferee is currently unavailable due to Python
version incompatibility.
# You may explore other libraries for coreference resolution like 'spaCy-pronom'
(check compatibility)
# or models from 'Hugging Face Transformers'.
print("Coreference resolution with coreferee skipped due to incompatibility with
Python 3.12.")
# if hasattr(doc, '_coref_clusters') and doc._coref_clusters:
#     for cluster in doc._coref_clusters:
#         print(cluster)
# else:
#     print("No coreference clusters found.")
```



```
... Original Text: John met Mary. He told her that he was going to the store. She smiled at him.  
Coreference Clusters (coreferee):  
Coreference resolution with coreferee skipped due to incompatibility with Python 3.12.
```

2. TEXT VECTORIZATION AND SIMILARITY

a. Bag of words(BOW) vectorization and representation

Input:

```
import pandas as pd  
from sklearn.feature_extraction.text import CountVectorizer  
  
# Sample text data  
corpus = [  
    'This is the first document.',  
    'This document is the second document.',  
    'And this is the third one.',  
    'Is this the first document?'  
]  
  
print("Original Corpus:")  
for i, doc in enumerate(corpus):  
    print(f"Document {i+1}: {doc}")
```

Output:

```
... Original Corpus:  
Document 1: This is the first document.  
Document 2: This document is the second document.  
Document 3: And this is the third one.  
Document 4: Is this the first document?
```

Input:

```
# 1. Initialize the CountVectorizer  
# It converts a collection of text documents to a matrix of token counts.  
# stop_words='english' removes common words like 'is', 'the', 'a' etc.  
vectorizer = CountVectorizer(stop_words='english')  
  
# 2. Fit the vectorizer to the corpus and transform the documents into BoW vectors  
X = vectorizer.fit_transform(corpus)
```

```
# Get the feature names (i.e., the vocabulary)
vocabulary = vectorizer.get_feature_names_out()

print("\nVocabulary (unique words after removing stop words):")
print(vocabulary)
```

Output:

```
...
Vocabulary (unique words after removing stop words):
['document' 'second']
```

Input:

```
# 3. Display the Bag of Words representation
# The sparse matrix X is converted to a dense array for easier viewing
bow_matrix = X.toarray()

# Create a DataFrame for better visualization
df_bow = pd.DataFrame(bow_matrix, columns=vocabulary)

print("\nBag of Words Representation (Document-Term Matrix):")
display(df_bow)

print("\nInterpretation:")
print("Each row represents a document, and each column represents a word from the vocabulary.")
print("The value in each cell indicates the count of that word in the corresponding document.")
```

Output:

```
...
Bag of Words Representation (Document-Term Matrix):
```

	document	second
0	1	0
1	2	1
2	0	0
3	1	0

```

Interpretation:
Each row represents a document, and each column represents a word from the vocabulary.
The value in each cell indicates the count of that word in the corresponding document.
```

b. TF-IDF implementation and comparison with BOW

Input:

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
# 1. Initialize the TfidfVectorizer
```

```
# Similar to CountVectorizer, we can remove stop words.
```

```
tfidf_vectorizer = TfidfVectorizer(stop_words='english')
```

```
# 2. Fit the vectorizer to the corpus and transform the documents into TF-IDF vectors
```

```
X_tfidf = tfidf_vectorizer.fit_transform(corpus)
```

```
# Get the feature names (i.e., the vocabulary, which should be the same as BoW if stop
words are consistent)
```

```
vocabulary_tfidf = tfidf_vectorizer.get_feature_names_out()
```

```
print("\nVocabulary (unique words after removing stop words for TF-IDF):")
```

```
print(vocabulary_tfidf)
```

Output:

```
...
Vocabulary (unique words after removing stop words for TF-IDF):
['document' 'second']
```

Input:

```
# 3. Display the TF-IDF representation
```

```
# Convert the sparse matrix X_tfidf to a dense array for easier viewing
```

```
tfidf_matrix = X_tfidf.toarray()
```

```
# Create a DataFrame for better visualization
df_tfidf = pd.DataFrame(tfidf_matrix, columns=vocabulary_tfidf)

print("\nTF-IDF Representation (Document-Term Matrix):")
display(df_tfidf)

print("\nInterpretation:")
print("Each row represents a document, and each column represents a word from the vocabulary.")
print("The value in each cell is the TF-IDF score for that word in the corresponding document.")
```

Output:

```
...
TF-IDF Representation (Document-Term Matrix):
   document second
0  1.000000  0.000000
1  0.787223  0.616668
2  0.000000  0.000000
3  1.000000  0.000000

Interpretation:
Each row represents a document, and each column represents a word from the vocabulary.
The value in each cell is the TF-IDF score for that word in the corresponding document.
```

c. N-gram models(uni, bi, and trigram) from the corpus

Input:

1. Unigram Model (N=1)

```
# Initialize CountVectorizer for unigrams (ngram_range=(1,1))
unigram_vectorizer = CountVectorizer(ngram_range=(1,1), stop_words='english')

# Fit and transform the corpus
X_unigram = unigram_vectorizer.fit_transform(corpus)

# Get feature names (vocabulary)
vocabulary_unigram = unigram_vectorizer.get_feature_names_out()

print("\nUnigram Vocabulary:")
print(vocabulary_unigram)
```

```
# Display the Unigram representation
df_unigram = pd.DataFrame(X_unigram.toarray(), columns=vocabulary_unigram)
print("\nUnigram Document-Term Matrix:")
display(df_unigram)
```

Output:

```
***
Unigram Vocabulary:
['document' 'second']

Unigram Document-Term Matrix:
   document  second
0         1         0
1         2         1
2         0         0
3         1         0
```

Input:

2. Bigram Model (N=2)

```
# Initialize CountVectorizer for bigrams (ngram_range=(2,2))
bigram_vectorizer = CountVectorizer(ngram_range=(2,2), stop_words='english')

# Fit and transform the corpus
X_bigram = bigram_vectorizer.fit_transform(corpus)

# Get feature names (vocabulary)
vocabulary_bigram = bigram_vectorizer.get_feature_names_out()

print("\nBigram Vocabulary:")
print(vocabulary_bigram)

# Display the Bigram representation
df_bigram = pd.DataFrame(X_bigram.toarray(), columns=vocabulary_bigram)
print("\nBigram Document-Term Matrix:")
display(df_bigram)
```

Output:

```
...
Bigram Vocabulary:
['document second' 'second document']

Bigram Document-Term Matrix:
      document second  second document
0          0          0          0
1          1          1          1
2          0          0          0
3          0          0          0
```

Input:

3. Trigram Model (N=3)

```
# Initialize CountVectorizer for trigrams (ngram_range=(3,3))
trigram_vectorizer = CountVectorizer(ngram_range=(3,3), stop_words='english')
```

```
# Fit and transform the corpus
X_trigram = trigram_vectorizer.fit_transform(corpus)
```

```
# Get feature names (vocabulary)
vocabulary_trigram = trigram_vectorizer.get_feature_names_out()
```

```
print("\nTrigram Vocabulary:")
print(vocabulary_trigram)
```

```
# Display the Trigram representation
df_trigram = pd.DataFrame(X_trigram.toarray(), columns=vocabulary_trigram)
print("\nTrigram Document-Term Matrix:")
display(df_trigram)
```

Output:

```
...
Trigram Vocabulary:
['document second document']

Trigram Document-Term Matrix:
      document second document
0          0          0          0
1          1          1          1
2          0          0          0
3          0          0          0
```

d. Cosine similarity computations between text document

Input:

```
import numpy as np

print("\n--- Angle (Theta) for Bag of Words (BoW) Representation ---")
# Calculate theta (angle in radians) from cosine similarity
theta_bow_radians = np.arccos(bow_similarity_matrix)

# Convert radians to degrees
theta_bow_degrees = np.degrees(theta_bow_radians)

# Create a DataFrame for better visualization
df_theta_bow_degrees = pd.DataFrame(theta_bow_degrees, index=[f'Doc {i+1}' for i
in range(len(corpus))], columns=[f'Doc {i+1}' for i in range(len(corpus))])

print("Angle Matrix (BoW) in Degrees:")
display(df_theta_bow_degrees)

print("\nInterpretation (BoW):")
print("Values close to 0 degrees indicate high similarity (vectors are nearly aligned).
Values close to 90 degrees indicate no similarity (vectors are orthogonal).")
print("Notice that Document 1 and Document 4 have an angle of 0 degrees, indicating
perfect alignment and thus identical content. Document 3 has an angle of 90 degrees
with all other documents, confirming their lack of shared terms.")
```

Output:

```
---
--- Angle (Theta) for Bag of Words (BoW) Representation ---
Angle Matrix (BoW) in Degrees:

```

	Doc 1	Doc 2	Doc 3	Doc 4
Doc 1	0.000000	2.656505e+01	90.0	0.000000
Doc 2	26.565051	8.537736e-07	90.0	26.565051
Doc 3	90.000000	9.000000e+01	90.0	90.000000
Doc 4	0.000000	2.656505e+01	90.0	0.000000

```

Interpretation (BoW):
Values close to 0 degrees indicate high similarity (vectors are nearly aligned). Values close to 90 degrees indicate no similarity (vectors are orthogonal).
Notice that Document 1 and Document 4 have an angle of 0 degrees, indicating perfect alignment and thus identical content. Document 3 has an angle of 90 degrees
with all other documents
```

Input:

```
print("\n--- Angle (Theta) for TF-IDF Representation ---")
# Calculate theta (angle in radians) from cosine similarity
theta_tfidf_radians = np.arccos(tfidf_similarity_matrix)
```

```

# Convert radians to degrees
theta_tfidf_degrees = np.degrees(theta_tfidf_radians)

# Create a DataFrame for better visualization
df_theta_tfidf_degrees = pd.DataFrame(theta_tfidf_degrees, index=[f'Doc {i+1}' for i
in range(len(corpus))], columns=[f'Doc {i+1}' for i in range(len(corpus))])

print("Angle Matrix (TF-IDF) in Degrees:")
display(df_theta_tfidf_degrees)

print("\nInterpretation (TF-IDF):")
print("Similar to BoW, 0 degrees means perfect similarity, and 90 degrees means no
similarity. The angles here will reflect the slightly different similarity scores observed
with TF-IDF due to its term weighting.")
print("For example, the angle between Document 1 and Document 2 is now larger
(indicating less similarity) compared to the BoW calculation, which aligns with the
lower cosine similarity score we saw for TF-IDF.")

```

Output:

```

...
--- Angle (Theta) for TF-IDF Representation ---
Angle Matrix (TF-IDF) in Degrees:

```

	Doc 1	Doc 2	Doc 3	Doc 4
Doc 1	0.000000	38.073254	90.0	0.000000
Doc 2	38.073254	0.000000	90.0	38.073254
Doc 3	90.000000	90.000000	90.0	90.000000
Doc 4	0.000000	38.073254	90.0	0.000000

```

Interpretation (TF-IDF):
Similar to BoW, 0 degrees means perfect similarity, and 90 degrees means no similarity.
For example, the angle between Document 1 and Document 2 is now larger (indicating less

```

Input:

```

from sklearn.metrics.pairwise import cosine_similarity

print("\n--- Cosine Similarity for Bag of Words (BoW) Representation ---")
# Compute cosine similarity between documents based on BoW
# Convert DataFrame to a NumPy array for cosine_similarity function
bow_similarity_matrix = cosine_similarity(df_bow.values)

# Create a DataFrame for better visualization

```



```
df_bow_similarity = pd.DataFrame(bow_similarity_matrix, index=[f'Doc {i+1}' for i
in range(len(corpus))], columns=[f'Doc {i+1}' for i in range(len(corpus))])
```

```
print("Cosine Similarity Matrix (BoW):")
display(df_bow_similarity)
```

```
print("\nInterpretation (BoW):")
print("Values close to 1 indicate high similarity, while values close to 0 indicate low
similarity.")
print("For example, Document 1 and Document 4 are identical in terms of their
'document' word count and lack of 'second', resulting in a perfect similarity of 1.0.
Document 2, with 'second' and more 'document' counts, shows less similarity to others
than Document 1 and 4 do to each other.")
```

Output:

```
--- Cosine Similarity for Bag of Words (BoW) Representation ---
Cosine Similarity Matrix (BoW):
```

	Doc 1	Doc 2	Doc 3	Doc 4
Doc 1	1.000000	0.894427	0.0	1.000000
Doc 2	0.894427	1.000000	0.0	0.894427
Doc 3	0.000000	0.000000	0.0	0.000000
Doc 4	1.000000	0.894427	0.0	1.000000

```

Interpretation (BoW):
Values close to 1 indicate high similarity, while values close to 0 indicate low similarity.
For example, Document 1 and Document 4 are identical in terms of their 'document' word count and lack of 'second', resulting in a perfect similarity of 1.0. Document 2

```

Input:

```
print("\n--- Cosine Similarity for TF-IDF Representation ---")
# Compute cosine similarity between documents based on TF-IDF
tfidf_similarity_matrix = cosine_similarity(df_tfidf.values)
```

```
# Create a DataFrame for better visualization
```

```
df_tfidf_similarity = pd.DataFrame(tfidf_similarity_matrix, index=[f'Doc {i+1}' for i
in range(len(corpus))], columns=[f'Doc {i+1}' for i in range(len(corpus))])
```

```
print("Cosine Similarity Matrix (TF-IDF):")
display(df_tfidf_similarity)
```

```
print("\nInterpretation (TF-IDF):")
print("Similar to BoW, values close to 1 indicate high similarity. Notice how the
similarity scores might differ slightly from BoW because TF-IDF weights words
based on their importance across the corpus, not just their raw frequency.")
```

print("Document 1 and Document 4 still show perfect similarity because their content is identical after stop word removal, and thus their TF-IDF vectors are the same. Document 2, which contains the unique word 'second', stands out more, and its similarity to other documents is adjusted by the inverse document frequency of 'document' and 'second'.")

Output:

```
--- Cosine Similarity for TF-IDF Representation ---
Cosine Similarity Matrix (TF-IDF):
```

	Doc 1	Doc 2	Doc 3	Doc 4
Doc 1	1.000000	0.787223	0.0	1.000000
Doc 2	0.787223	1.000000	0.0	0.787223
Doc 3	0.000000	0.000000	0.0	0.000000
Doc 4	1.000000	0.787223	0.0	1.000000

```
Interpretation (TF-IDF):
Similar to BoW, values close to 1 indicate high similarity. Notice how the similarity scores might differ slightly from BoW because TF-IDF
Document 1 and Document 4 still show perfect similarity because their content is identical after stop word removal, and thus their TF-IDF \
```

e. Word2Vec Word Embeddings using Gensim on a Custom Corpus

Input:

```
!pip install gensim nltk
import nltk
from nltk.tokenize import word_tokenize
from gensim.models import Word2Vec
import pandas as pd

nltk.download('punkt')
nltk.download('punkt_tab') # Explicitly download punkt_tab
corpus = [
    "AI improves systems",
    "Artificial intelligence enhances systems",
    "Machine learning improves AI",
    "AI handles complex problems",
    "Deep learning is part of artificial intelligence"
]

print("Original Corpus:\n", corpus)
tokenized_corpus = [word_tokenize(sentence.lower()) for sentence in corpus]

print("\nTokenized Corpus:")
print(tokenized_corpus)
```

```

model = Word2Vec(
    sentences=tokenized_corpus,
    vector_size=50,
    window=2,
    min_count=1,
    workers=4
)

print("\nVocabulary Words:")
print(list(model.wv.index_to_key))

print("\nWord Vector for 'ai':")
print(model.wv['ai'])

print("\nWords similar to 'ai':")
print(model.wv.most_similar('ai'))

similarity = model.wv.similarity('ai', 'learning')
print("\nSimilarity between 'ai' and 'learning':")
print(similarity)

model.save("word2vec_model.model")

print("\nModel saved successfully!")

```

Output:

```

*** Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.2)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
Original Corpus:
['AI improves systems', 'Artificial intelligence enhances systems', 'Machine learning improves AI', 'AI handles complex problems', 'Deep learning is part of artificial in
Tokenized Corpus:
[['ai', 'improves', 'systems'], ['artificial', 'intelligence', 'enhances', 'systems'], ['machine', 'learning', 'improves', 'ai'], ['ai', 'handles', 'complex', 'problems'],
Vocabulary Words:
['ai', 'learning', 'intelligence', 'artificial', 'systems', 'improves', 'of', 'part', 'is', 'deep', 'problems', 'complex', 'handles', 'machine', 'enhances']

Word Vector for 'ai':
[-1.0724545e-03  4.7286271e-04  1.0206699e-02  1.8018546e-02
 -1.8605899e-02 -1.4233618e-02  1.2917745e-02  1.7945977e-02
 -1.0030856e-02 -7.5267432e-03  1.4761089e-02 -3.0669428e-03
 -9.072267e-03  1.3108194e-02 -5.7201208e-03 -3.6320353e-03
 5.751595e-03  1.9837476e-03 -1.6570430e-02 -1.8897636e-02
 1.4623532e-02  1.0140524e-02  1.3515387e-02  1.5257311e-03
 1.2701781e-02 -6.8107117e-03 -1.8028028e-03  1.1537147e-02
 -1.5043275e-02 -7.8722071e-03 -1.5023164e-02 -1.8600845e-03
 1.9076237e-02 -1.4638334e-02 -4.6675373e-03 -3.8754821e-03
 1.6154874e-02 -1.1861792e-02  9.0324880e-05 -9.5074680e-03
 -1.9207101e-02  1.0014586e-02 -1.7519170e-02 -8.7836506e-03
 -7.0199967e-05 -5.9236289e-04 -1.5322480e-02  1.9229487e-02
 9.9641159e-03  1.8466286e-02]

Words similar to 'ai':
[('enhances', 0.2105710357427597), ('complex', 0.16704076528549194), ('machine', 0.15020693838596344), ('part', 0.13204213976860046), ('intelligence', 0.1267007291316986),
Similarity between 'ai' and 'learning':
0.042373005

Model saved successfully!

```

f. GloVe Embeddings loading and vector representation

Input:

```
import numpy as np

# Step 0: Download and unzip GloVe embeddings if not already present
# These files are typically hosted by Stanford NLP
!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip -q glove.6B.zip

# Step 1: Load GloVe file
glove_file = "glove.6B.50d.txt"

# Step 2: Create dictionary to store embeddings
embeddings_index = {}

# Step 3: Read file and store word vectors
with open(glove_file, 'r', encoding='utf8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = [float(val) for val in values[1:]]
        embeddings_index[word] = vector

print("Total words loaded:", len(embeddings_index))

# Step 4: Get vector representation of a word
word = "king"

if word in embeddings_index:
    print("Vector for word:", word)
    print(embeddings_index[word])
else:
    print("Word not found in embeddings")

# Step 5: Example similarity operation

king = np.array(embeddings_index["king"])
man = np.array(embeddings_index["man"])
woman = np.array(embeddings_index["woman"])
```

```

result = king - man + woman
print("Vector result of king - man + woman:")
print(result)

```

Output:

```

--- --2026-04-15 17:12:53-- http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-04-15 17:12:53-- https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-04-15 17:12:53-- https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: 'glove.6B.zip'

glove.6B.zip      100%[=====>] 822.24M  4.99MB/s   in 2m 40s

2026-04-15 17:15:34 (5.14 MB/s) - 'glove.6B.zip' saved [862182613/862182613]

Total words loaded: 400000
Vector for word: king
[0.50451, 0.68697, 0.59517, -0.022801, 0.60046, -0.13498, -0.08813, 0.47377, -0.61798, -0.31012, -0.076666,
Vector result of king - man + woman:
[ 0.417366  0.90427  -1.00503  -0.062021  0.49726  0.80667
 -0.14855  0.80365  -0.15654  -0.66974  0.234354  0.62476
 0.925871  -0.971   0.92566  0.89915  -1.54596  -0.52625
 0.136954  0.66199  0.48716  0.37035  -0.214214  0.10101
 0.71358  -2.0875  -1.1362  -1.14961  -0.53599  0.2739
 1.6723   0.02931  -0.77656  0.46056286  0.34866  -0.057417
 0.19444  -0.207748  -0.73039  -0.10752  0.235544  0.96424
 -0.46994  -0.487275  -0.254   0.46213  -0.66081  -1.94515
 -0.68797  -0.49784  ]

```

g. Text Similarity using Word Mover's Distance (WMD)

Input:

```
import numpy as np
```

```
# Step 0: Download and unzip GloVe embeddings if not already present
```

```
# These files are typically hosted by Stanford NLP
```

```
!wget http://nlp.stanford.edu/data/glove.6B.zip
```

```
!unzip -q glove.6B.zip
```

```
# Step 1: Load GloVe file
```

```
glove_file = "glove.6B.50d.txt"
```

```
# Step 2: Create dictionary to store embeddings
```

```
embeddings_index = {}
```

```
# Step 3: Read file and store word vectors
```

```
with open(glove_file, 'r', encoding='utf8') as f:
```

```
    for line in f:
```

```
        values = line.split()
```

```
        word = values[0]
```

```

        vector = [float(val) for val in values[1:]]
        embeddings_index[word] = vector

print("Total words loaded:", len(embeddings_index))

# Step 4: Get vector representation of a word
word = "king"

if word in embeddings_index:
    print("Vector for word:", word)
    print(embeddings_index[word])
else:
    print("Word not found in embeddings")

# Step 5: Example similarity operation

king = np.array(embeddings_index["king"])
man = np.array(embeddings_index["man"])
woman = np.array(embeddings_index["woman"])

result = king - man + woman

print("Vector result of king - man + woman:")
print(result)

```

Output:

```

*** --2026-04-15 17:12:53-- http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)[171.64.67.140]:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-04-15 17:12:53-- https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)[171.64.67.140]:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-04-15 17:12:53-- https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)[171.64.64.22]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: 'glove.6B.zip'

glove.6B.zip      100%[=====] 822.24M  4.99MB/s   in 2m 40s

2026-04-15 17:15:34 (5.14 MB/s) - 'glove.6B.zip' saved [862182613/862182613]

Total words loaded: 400000
Vector for word: king
[0.50451, 0.68607, -0.59517, -0.022801, 0.60046, -0.13498, -0.08813, 0.47377, -0.61798, -0.31012, -0.076666, 1.493,
Vector result of king - man + woman:
[ 0.417366  0.90427  -1.00503  -0.062021  0.49726  0.80667
 -0.14855  0.80365  -0.15654  -0.66974  0.234354  0.62476
 0.925871  -0.971   0.92566  0.89915  -1.54596  -0.52625
 0.136954  0.66199  0.48716  0.37035  -0.214214  0.10101
 0.71358  -2.0875  -1.1362  -1.14961  -0.53599  0.2739
 1.6723   0.02931  -0.77656  0.46056286  0.34866  -0.057417
 0.19444  -0.207748  -0.73039  -0.10752  0.235544  0.96424
 -0.46994  -0.487275  -0.254   0.46213  -0.66081  -1.94515
 -0.68797  -0.49784  ]

```

Input:

```
!pip install POT
# Calculate WMD between all pairs of documents in the corpus
num_documents = len(tokenized_corpus)
wmd_matrix = np.zeros((num_documents, num_documents))

# Create a WmdSimilarity index. This is more efficient for comparing a query to many
documents.
# However, for all-pairs similarity, we can iterate directly.

print("\nCalculating Word Mover's Distance between document pairs...")

for i in range(num_documents):
    for j in range(num_documents):
        if i == j:
            wmd_matrix[i, j] = 0.0 # Distance to itself is 0
        else:
            # gensim's wmdistance returns the distance (lower is more similar)
            distance = model.wv.wmdistance(tokenized_corpus[i], tokenized_corpus[j])
            wmd_matrix[i, j] = distance

# Create a DataFrame for better visualization
df_wmd = pd.DataFrame(
    wmd_matrix,
    index=[f'Doc {i+1}' for i in range(num_documents)],
    columns=[f'Doc {i+1}' for i in range(num_documents)]
)

print("\nWord Mover's Distance Matrix:")
display(df_wmd)

print("\nInterpretation:")
print("Lower WMD values indicate higher semantic similarity between documents.
Values are distances, not similarity scores, so 0 means identical content
(semantically).")
print("Notice that documents with related concepts (e.g., 'AI', 'artificial intelligence',
'machine learning', 'deep learning') will have lower WMD values even if they don't
share many exact words, due to the underlying word embeddings.")
```

Output:

```
*** Collecting POT
  Downloading pot-0.9.6.post1-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)
    40.2/40.2 kB 2.5 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.16 in /usr/local/lib/python3.12/dist-packages (from POT) (2.0.2)
Requirement already satisfied: scipy>=1.6 in /usr/local/lib/python3.12/dist-packages (from POT) (1.16.3)
  Downloading pot-0.9.6.post1-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (1.5 MB)
    1.5/1.5 MB 31.6 MB/s eta 0:00:00

Installing collected packages: POT
Successfully installed POT-0.9.6.post1

Calculating Word Mover's Distance between document pairs...

Word Mover's Distance Matrix:

```

	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5
Doc 1	0.000000	1.032129	0.645935	1.031000	1.321784
Doc 2	1.032129	0.000000	1.371692	1.260845	0.965240
Doc 3	0.645935	1.371692	0.000000	0.999194	1.162289
Doc 4	1.031000	1.260845	0.999194	0.000000	1.307533
Doc 5	1.321784	0.965240	1.162289	1.307533	0.000000

```

Interpretation:
Lower WMD values indicate higher semantic similarity between documents. Values are distances, not similarity scores, so 0 means identical content (semantically).
Notice that documents with related concepts (e.g., 'AI', 'artificial intelligence', 'machine learning', 'deep learning') will have lower WMD values even if they d

```

3. TEXT ANALYTICS

a. Text Classification using Naïve Bayes/SVM with TF-IDF

Input:

Step 1: Import libraries

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report

Step 2: Create sample dataset

data = {

 "text": [

 "I love this product",

 "This phone is amazing",

 "The movie was terrible",

 "I hate this service",

 "The product quality is great",

 "This is a bad experience",

 "Fantastic phone and great battery",

 "Worst customer support ever"


```

],
"label": [
    "positive",
    "positive",
    "negative",
    "negative",
    "positive",
    "negative",
    "positive",
    "negative"
]
}

df = pd.DataFrame(data)
print("Dataset:\n", df)

# Step 3: Split dataset into train and test
X_train, X_test, y_train, y_test = train_test_split(
    df["text"],
    df["label"],
    test_size=0.3,
    random_state=42
)

# Step 4: Convert text to TF-IDF features
tfidf = TfidfVectorizer()

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("\nTF-IDF Feature Shape:", X_train_tfidf.shape)
nb_model = MultinomialNB()

nb_model.fit(X_train_tfidf, y_train)

nb_predictions = nb_model.predict(X_test_tfidf)

print("\nNaïve Bayes Predictions:")
print(nb_predictions)

print("\nNaïve Bayes Accuracy:")

```

```

print(accuracy_score(y_test, nb_predictions))
svm_model = SVC(kernel="linear")
svm_model.fit(X_train_tfidf, y_train)
svm_predictions = svm_model.predict(X_test_tfidf)

print("\nSVM Predictions:")
print(svm_predictions)

print("\nSVM Accuracy:")
print(accuracy_score(y_test, svm_predictions))

print("\nClassification Report (SVM):")
print(classification_report(y_test, svm_predictions))

```

Output:

```

Dataset:
...
      text      label
0  I love this product  positive
1  This phone is amazing  positive
2  The movie was terrible  negative
3  I hate this service  negative
4  The product quality is great  positive
5  This is a bad experience  negative
6  Fantastic phone and great battery  positive
7  Worst customer support ever  negative

TF-IDF Feature Shape: (5, 19)

Naïve Bayes Predictions:
['negative' 'negative' 'negative']

Naïve Bayes Accuracy:
0.3333333333333333

SVM Predictions:
['negative' 'negative' 'negative']

SVM Accuracy:
0.3333333333333333

Classification Report (SVM):
      precision    recall  f1-score   support

 negative      0.33      1.00      0.50         1
 positive      0.00      0.00      0.00         2

 accuracy      0.17      0.50      0.25         3
 macro avg      0.17      0.50      0.25         3
 weighted avg      0.11      0.33      0.17         3

```

b. Sentiment Analysis using TextBlob and VADER

Input:

Step 1: Install required libraries

!pip install textblob

!pip install vaderSentiment

Step 2: Import libraries

from textblob import TextBlob

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

```

# Step 3: Example text data
sentences = [
    "I love this phone, it works perfectly!",
    "This product is terrible and I hate it.",
    "The service was okay, not great but not bad.",
    "Amazing experience! Highly recommended.",
    "Worst purchase ever."
]

print("Sentiment Analysis using TextBlob:\n")

for sentence in sentences:
    blob = TextBlob(sentence)
    polarity = blob.sentiment.polarity
    subjectivity = blob.sentiment.subjectivity
    print("Sentence:", sentence)
    print("Polarity:", polarity)
    print("Subjectivity:", subjectivity)
    print()

print("\nSentiment Analysis using VADER:\n")
analyzer = SentimentIntensityAnalyzer()
for sentence in sentences:
    score = analyzer.polarity_scores(sentence)
    print("Sentence:", sentence)
    print("VADER Scores:", score)
    print()

```

Output:

```

Sentiment Analysis using VADER:

Sentence: I love this phone, it works perfectly!
VADER Scores: {'neg': 0.0, 'neu': 0.365, 'pos': 0.635, 'compound': 0.8655}

Sentence: This product is terrible and I hate it.
VADER Scores: {'neg': 0.531, 'neu': 0.469, 'pos': 0.0, 'compound': -0.7783}

Sentence: The service was okay, not great but not bad.
VADER Scores: {'neg': 0.161, 'neu': 0.449, 'pos': 0.391, 'compound': 0.4728}

Sentence: Amazing experience! Highly recommended.
VADER Scores: {'neg': 0.0, 'neu': 0.244, 'pos': 0.756, 'compound': 0.7339}

Sentence: Worst purchase ever.
VADER Scores: {'neg': 0.672, 'neu': 0.328, 'pos': 0.0, 'compound': -0.6249}

```

c. Topic Modeling using Latent Dirichlet Allocation (LDA)

Input:

```
# Step 1: Install required library
!pip install gensim

# Step 2: Import libraries
import gensim
from gensim import corpora
from gensim.models import LdaModel
from gensim.utils import simple_preprocess

# Step 3: Create sample documents
documents = [
    "Artificial intelligence and machine learning are transforming technology",
    "Deep learning models improve AI systems",
    "Football and cricket are popular sports",
    "The team won the football championship",
    "AI and data science are related fields"
]

# Step 4: Text preprocessing (tokenization)
tokenized_docs = [simple_preprocess(doc) for doc in documents]
print("Tokenized Documents:")
print(tokenized_docs)

# Step 5: Create dictionary
dictionary = corpora.Dictionary(tokenized_docs)

print("\nDictionary:")
print(dictionary.token2id)

# Step 6: Create corpus (Bag-of-Words format)
corpus = [dictionary.doc2bow(text) for text in tokenized_docs]

print("\nCorpus (BoW representation):")
print(corpus)

# Step 7: Train LDA model
lda_model = LdaModel(
    corpus=corpus,
    id2word=dictionary,
```

)

Installing colli

```

Installing collected packages: gensim
Successfully installed gensim-4.4.0
Tokenized Documents:
[['artificial', 'intelligence', 'and', 'machine', 'learning', 'are', 'transforming', 'technology'], ['deep', 'learning', 'models', 'improve', 'ai', 'systems'], ['football', 'and', 'cricket']]

Dictionary:
{'and': 0, 'are': 1, 'artificial': 2, 'intelligence': 3, 'learning': 4, 'machine': 5, 'technology': 6, 'transforming': 7, 'ai': 8, 'deep': 9, 'improve': 10, 'models': 11, 'systems': 12, 'cricket': 13}

Corpus (BoW representation):
[[([0, 1], (1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1)), ([4, 1], (8, 1), (9, 1), (10, 1), (11, 1), (12, 1)), ([0, 1], (1, 1), (13, 1), (14, 1), (15, 1), (16, 1)), [(14, 1), (17, 1)]]

Topics discovered by LDA:
Topic 0: 0.075**learning" + 0.074**and" + 0.074**are" + 0.072**transforming" + 0.072**machine" + 0.072**intelligence" + 0.072**artificial" + 0.072**technology" + 0.025**ai" + 0.025**systems
Topic 1: 0.067**the" + 0.067**football" + 0.067**ai" + 0.066**are" + 0.066**and" + 0.040**won" + 0.040**championship" + 0.040**team" + 0.040**data" + 0.040**fields

```

d. Topic Modeling using Latent Semantic Analysis (LSA)

Input:

```
!pip install gensim
```

```
from gensim import corpora
from gensim.models import LsiModel
from gensim.utils import simple_preprocess
```

```
documents = [
    "Artificial intelligence and machine learning are transforming technology",
    "Deep learning models improve AI systems",
    "Football and cricket are popular sports",
    "The team won the football championship",
    "AI and data science are related fields"
]
```

```
tokenized_docs = [simple_preprocess(doc) for doc in documents]
```

```
print("Tokenized Documents:")
```

```

print(tokenized_docs)

dictionary = corpora.Dictionary(tokenized_docs)

print("\nDictionary:")
print(dictionary.token2id)
corpus = [dictionary.doc2bow(text) for text in tokenized_docs]
print("\nCorpus (BoW representation):")
print(corpus)
lsa_model = LsiModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=2
)
print("\nTopics discovered by LSA:")
for idx, topic in lsa_model.print_topics():
    print(f"Topic {idx}: {topic}")

```

Output:

```

Tokenized Documents:
[['artificial', 'intelligence', 'and', 'machine', 'learning', 'are', 'transforming', 'technology'], ['deep', 'learning', 'models', 'improve', 'ai', 'systems'], ['football', 'the', 'team', 'won', 'championship']]

Dictionary:
{'and': 0, 'are': 1, 'artificial': 2, 'intelligence': 3, 'learning': 4, 'machine': 5, 'technology': 6, 'transforming': 7, 'ai': 8, 'deep': 9, 'improve': 10, 'models': 11, 'systems': 12, 'football': 13, 'the': 14, 'team': 15, 'won': 16, 'championship': 17}

Corpus (BoW representation):
[[([0, 1], (1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1)), [(4, 1), (8, 1), (9, 1), (10, 1), (11, 1), (12, 1)], [(0, 1), (1, 1), (13, 1), (14, 1), (15, 1), (16, 1)], [(14, 1), (15, 1), (16, 1), (17, 1)]]

Topics discovered by LSA:
Topic 0: 0.490*and + 0.490*are + 0.256*learning + 0.227*ai + 0.191*intelligence + 0.191*machine + 0.191*technology + 0.191*artificial + 0.191*transforming + 0.191*systems
Topic 1: 0.667*the + 0.393*football + 0.333*team + 0.333*won + 0.333*championship + -0.107*learning + -0.088*ai + -0.060*intelligence + -0.060*technology + -0.060*systems

```

e. Opinion Mining on product/service reviews dataset

Input:

```

!pip install textblob
import pandas as pd
from textblob import TextBlob

```

```

data = {
    "review": [
        "This phone is amazing and works perfectly",
        "The product quality is very bad",
    ]
}

```

```

        "I love the camera and battery life",
        "The service was terrible and disappointing",
        "Good value for money"
    ]
}

```

```

df = pd.DataFrame(data)
print("Dataset:")
print(df)
sentiments = []
for review in df["review"]:
    analysis = TextBlob(review)
    polarity = analysis.sentiment.polarity
    if polarity > 0:
        sentiment = "Positive"
    elif polarity < 0:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"
    sentiments.append(sentiment)
df["Sentiment"] = sentiments
print("\nOpinion Mining Results:")
print(df)

```

Output:

```

... Requirement already satisfied: textblob in /usr/local/lib/python3.12/dist-packages (0.19.0)
Requirement already satisfied: nltk<=3.9 in /usr/local/lib/python3.12/dist-packages (from textblob) (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk<=3.9->textblob) (8.3.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk<=3.9->textblob) (1.5.3)
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk<=3.9->textblob) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk<=3.9->textblob) (4.67.3)
Dataset:
   review
0  This phone is amazing and works perfectly
1  The product quality is very bad
2  I love the camera and battery life
3  The service was terrible and disappointing
4  Good value for money

Opinion Mining Results:
   review Sentiment
0  This phone is amazing and works perfectly Positive
1  The product quality is very bad Negative
2  I love the camera and battery life Positive
3  The service was terrible and disappointing Negative
4  Good value for money Positive

```

f. Information Extraction (IE) from structured/unstructured documents

Input:

```
!pip install spacy
```

```
!python -m spacy download en_core_web_sm
```

```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
text = """
```

```
Apple Inc. was founded by Steve Jobs and Steve Wozniak in 1976.
```

```
The company is headquartered in Cupertino, California.
```

```
Microsoft was founded by Bill Gates in 1975.
```

```
"""
```

```
print("Input Text:")
```

```
print(text)
```

```
doc = nlp(text)
```

```
print("\nExtracted Named Entities:\n")
```

```
for ent in doc.ents:
```

```
    print(ent.text, "->", ent.label_)
```

Output:

```
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.
```

```
Input Text:
```

```
Apple Inc. was founded by Steve Jobs and Steve Wozniak in 1976.
```

```
The company is headquartered in Cupertino, California.
```

```
Microsoft was founded by Bill Gates in 1975.
```

```
Extracted Named Entities:
```

```
Apple Inc. -> ORG
```

```
Steve Jobs -> PERSON
```

```
Steve Wozniak -> PERSON
```

```
1976 -> DATE
```

```
Cupertino -> GPE
```

```
California -> GPE
```

```
Microsoft -> ORG
```

```
Bill Gates -> PERSON
```

```
1975 -> DATE
```


g. Information Retrieval system with ranking using TF-IDF

Input:

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "Artificial intelligence and machine learning are transforming technology",
    "Deep learning improves AI systems",
    "Football and cricket are popular sports",
    "AI and data science are related fields",
    "Sports like football and cricket are played worldwide"
]

print("Documents:")
for i, doc in enumerate(documents):
    print(i+1, ":", doc)

vectorizer = TfidfVectorizer()
query_vector = vectorizer.transform([query])
similarity_scores = cosine_similarity(query_vector, tfidf_matrix)
scores = similarity_scores.flatten()
ranked_results = np.argsort(scores)[::-1]
print("\nRanked Documents:")
for idx in ranked_results:
    print("Document", idx+1, "Score:", scores[idx])
    print(documents[idx])
    print()
```

Output:

```
.. Documents:
 1 : Artificial intelligence and machine learning are transforming technology
 2 : Deep learning improves AI systems
 3 : Football and cricket are popular sports
 4 : AI and data science are related fields
 5 : Sports like football and cricket are played worldwide

TF-IDF Matrix Shape: (5, 23)

Query: AI and machine learning

Ranked Documents:
Document 1 Score: 0.4850990860842655
Artificial intelligence and machine learning are transforming technology

Document 2 Score: 0.3878298886897199
Deep learning improves AI systems

Document 4 Score: 0.2602424890261745
AI and data science are related fields

Document 3 Score: 0.10354304677903457
Football and cricket are popular sports

Document 5 Score: 0.08296772234937844
Sports like football and cricket are played worldwide
```

